🎯 Web Dev III — Complete Lab 1 Viva Notes & Solutions

Course: Web Dev III (Node.js & Express Backend) | **Unit:** Unit–1 Core Node.js
Assignment: Smart Utility Toolkit (CLI, CommonJS, HTTP Server, FS CRUD, Crypto)

📚 PART 1: CORE TOPIC NOTES & THEORY

1. Node.js Architecture & Runtime Environment

• **What it is:** A JavaScript runtime built on Google Chrome's **V8 engine** that runs JS on the server/operating system.

• **Key Characteristics:** Single-threaded, Event-driven, Asynchronous, Non-blocking I/O.

• **Core Modules Used:** `process` (environment/CLI), `http` (server), `fs` (file system), `crypto` (randomness/encryption).

2. CommonJS Module System (`module.exports` & `require`)

• **Exporting:** `module.exports = isEven;` or `module.exports = { isEven, isOdd };`

• **Importing:** `const isEven = require('./modules/isEven');` (Custom modules **require** relative path `./`, while core modules like `require('http')` do not).

3. Command Line Arguments (`process.argv`)

• `process.argv[0]` = Path to `node.exe`

• `process.argv[1]` = Path to currently running JS file

• `process.argv.slice(2)` = Actual user arguments array (strings: `['add', '10', '5']`).

4. File System Operations (`fs` Module)

• **Asynchronous vs Synchronous:** Async methods (e.g. `fs.writeFile`) are non-blocking and take a callback `(err, data) => {}`. Sync methods (e.g. `fs.writeFileSync`) block the thread.

• **CRUD Lifecycle:** Create (`writeFile`) → Read (`readFile`) → Update (`appendFile`) → Delete (`unlink`).

5. HTTP Server Routing & Crypto

• **HTTP Server:** `http.createServer((req, res) => { ... })`. Checks `req.url`, sets status via `res.writeHead(status)`, and finishes with `res.end(body)`.

• **Crypto Module:** `crypto.randomInt(min, max)` generates cryptographically secure uniform random integers (unlike `Math.random()` which is pseudo-random).

❓ PART 2: TOP VIVA QUESTIONS & CRISP SOLUTIONS

Q1. Why do we need `slice(2)` on `process.argv` in `calculator.js`?

Solution: The first two elements of `process.argv` are reserved for the Node runtime executable path and the script's filepath. User-supplied arguments only begin at index 2.

Q2. Why is `parseFloat()` or `Number()` mandatory before calculating?

Solution: All CLI arguments parsed from `process.argv` are of type `string`. Without conversion, `10 + 5` would result in string concatenation (`"105"`) instead of numeric addition (`15`).

Q3. What is the difference between `fs.readFile()` and `fs.readFileSync()`?

Solution: `fs.readFile()` is non-blocking (asynchronous); it delegates file reading to libuv thread pool and notifies via callback without freezing the Node server. `fs.readFileSync()` blocks the entire JavaScript thread until the file is read.

Q4. How do you prevent 'division by zero' in your calculator?

Solution: In `calculator.js`, when the operation is `div` or `/`, we check `if (num2 === 0)` and return an error message before dividing.

Q5. Why use `crypto.randomInt()` in `dice.js` instead of `Math.random()`?

Solution: `Math.random()` is pseudo-random and predictable. `crypto.randomInt()` uses hardware/system entropy to generate cryptographically strong, non-predictable random numbers.

Q6. What happens if you forget `res.end()` in `server.js`?

Solution: The HTTP response connection will stay open indefinitely. The browser client will keep spinning/loading until it hits a network timeout.

⚠️ TRICKY Q7. Why did you not use Express.js in this project?

Solution: *"Sir, the unit-1 lab guidelines strictly required us to use native Node.js core modules (`http`, `fs`, `process`, `crypto`) to understand low-level backend fundamentals before abstracting with Express.js."*

⚠️ TRICKY Q8. What is the difference between `module.exports` and `exports`?

Solution: `exports` is merely a reference pointing to `module.exports`. If you reassign `exports = myFunc`, the reference breaks. Assigning directly to `module.exports` is the recommended standard.

🚀 PART 3: EXACT TERMINAL COMMANDS TO RUN IN EXAM

Setup Step in Terminal: Open terminal in VS Code and ensure you are in the `Lecture 1` directory:

```
cd "C:\Users\acer\OneDrive\Desktop\Web dev -III assignemnts\Lecture 1"
```

Task / Question	Command to Type	Expected Output
1. Master Demo	`node index.js`	Shows banner + runs calculator & dice
2. Calculator (Add)	`node calculator.js add 10 5`	`Result: 15`
3. Calculator (Mult)	`node calculator.js mult 6 7`	`Result: 42`
4. Calculator (Div)	`node calculator.js div 100 4`	`Result: 25`
5. Custom Modules	`node app.js`	Demonstrates `isEven` & color logger
6. File Manager	`node fileManager.js`	Create → Read → Update → Delete
7. Dice Single	`node dice.js`	🎲 `Dice Rolled: 4`
8. Dice Multiple	`node dice.js 5`	Simulates 5 rolls + saves history
9. HTTP Server	`node server.js`	Open `http://localhost:3000` in browser

💡 PART 4: GOLDEN RULES TO SCORE 100% (2.5 / 2.5)

1. **Explain with Confidence:** State clearly which module is handling which task (e.g., "*We used* `process.argv` *for inputs,* `http` *for routing, and* `fs` *for async file operations*").

2. **Error Handling:** Point out that your code handles missing inputs, invalid routes (404), and division by zero gracefully.

3. **Mention Bonus Features:** If time permits, mention you added ANSI terminal coloring, timestamped logs, and dice history saving.